Pseudo-Journal of Low-Tech Cryptography for a High-Tech Threat

OSCAIR reviewed (OSCAIR review report can be found here)

The Quantum Computer May Decrypt the Pieces, But Not Necessarily the Puzzle

Compression, Fragmentation, and Ordering-Information Loss as a Post-Quantum Resilience Mechanism Using Old Standard Cryptographic Tools

Etienne Lemaire

Abstract

Quantum computing poses a significant challenge to several classes of cryptographic systems and has motivated the development of post-quantum cryptography. Most current approaches focus on replacing vulnerable cryptographic primitives with algorithms believed to be resistant to quantum attacks. This paper explores an orthogonal and complementary approach based not on new cryptographic primitives, but on the protection of the structural organization of information. We introduce a framework in which data security relies not only on encryption, but also on compression, fragmentation, deterministic permutation, and the concealment of reconstruction-critical information. The central idea is that recovering individual fragments may not be sufficient to recover the original message. In particular, when a compressed data stream depends on a compression dictionary whose location and reconstruction context are hidden through fragmentation and permutation, access to decrypted fragments does not necessarily provide access to usable information due to the sequence-dependent nature of data compression. A proof-of-concept implementation named PuzzleCrypt is presented. PuzzleCrypt combines standard cryptographic tools, data compression, block fragmentation, deterministic key-derived permutations, and multi-layer encryption to create a combined decryption-and-reconstruction problem. The objective is not to replace post-quantum cryptographic schemes, nor to claim resistance against future quantum computers, but rather to investigate whether reconstruction complexity can remain a meaningful obstacle even in scenarios where some cryptographic layers are weakened. More generally, this work explores and may provide an additional resilience layer for old systems.

Keywords: Post-Quantum Cryptography, Data Fragmentation, Compression, AES, Information Ordering, Reconstruction Complexity, Structural Security, PuzzleCrypt.

1. Introduction

Quantum computing has emerged as a major challenge for modern cryptography. Since Shor demonstrated that a sufficiently large quantum computer could efficiently solve integer factorization and discrete logarithm problems, widely deployed public-key cryptosystems such as RSA and elliptic curve cryptography (ECC) have been considered vulnerable in a post-quantum scenario [1], [2]. Symmetric cryptographic primitives are generally believed to be more resistant, although Grover's algorithm may provide a quadratic speedup for exhaustive key-search attacks, effectively reducing the security level of symmetric keys by approximately one half [3], [4].

In response to these developments, a substantial research effort has focused on Post-Quantum Cryptography (PQC), leading to new families of cryptographic algorithms based on lattice, code, hash, and multivariate mathematical problems [5]. These approaches primarily seek to preserve confidentiality by replacing vulnerable cryptographic primitives with quantum-resistant alternatives.

This paper explores an orthogonal and complementary approach. Rather than relying exclusively on stronger mathematical assumptions, we investigate whether the structural organization of information itself can provide an additional layer of resilience. The central hypothesis is that recovering encrypted fragments may not necessarily be equivalent to recovering the original message. In particular, when data are compressed, fragmented, permuted, and protected by multiple encryption layers, recovering individual fragments may still leave an attacker without the information required to reconstruct a valid and interpretable data stream.

The proposed approach is motivated by the observation that many compression schemes rely on reconstruction context, ordering information, and dictionary-dependent decoding processes. Previous research has demonstrated that compression contexts can leak information through side channels such as CRIME and BREACH [6], [7]. In contrast, this work investigates whether compression context can instead become a protected asset. More specifically, if the dictionary, the beginning of the compressed stream, or other reconstruction-critical blocks are hidden, fragmented, or protected by additional encryption layers, access to decrypted fragments may remain insufficient to recover meaningful information.

This concept can be viewed as a form of structural protection. Traditional cryptographic security focuses on protecting ciphertexts and cryptographic keys. PuzzleCrypt introduces the additional notion of **ordering information**, defined here as the information required to reconstruct a valid interpretation of a fragmented data stream. Such information may include fragment ordering, reconstruction context, dictionary location, fragment relationships, or hidden starting positions within the compressed stream.

The idea shares similarities with several existing research directions. Secret-sharing mechanisms distribute recovery information across multiple shares [8], while Information Dispersal Algorithms distribute data across multiple fragments for reliability and availability [9]. All-Or-Nothing Transforms seek to make partial recovery ineffective [10], and distributed secure storage systems such as Tahoe-LAFS combine encryption and fragmentation to improve confidentiality and

resilience [11]. However, to the best of our knowledge, no previous work has explicitly investigated the protection of ordering information and compression-context location as an independent security layer intended to increase reconstruction complexity.

Based on these observations, we introduce **PuzzleCrypt**, a proof-of-concept system combining standard cryptographic tools, compression, fragmentation, deterministic key-derived permutation, and multi-layer encryption. The objective is not to replace post-quantum cryptography, nor to claim resistance against future quantum attacks. Instead, PuzzleCrypt serves as an experimental framework for studying whether deliberate loss of structural context can transform a pure decryption problem into a combined decryption-and-reconstruction problem. The central research question investigated in this work can be summarized as follows:

A future attacker may be able to decrypt the pieces, but not necessarily solve the puzzle.

2. Puzzle-Based Data Protection on Compressed data

The original file is first encrypted and then fragmented into multiple pieces that are deterministically permuted using a secret key. Even if an attacker were able to decrypt individual pieces, reconstructing the original message would still require recovering the correct ordering and structural relationships between those pieces. Then a second layer of encryption comes and encrypt the whole encrypted puzzle with a second secret key. This initial idea is illustrated in Figure 1.

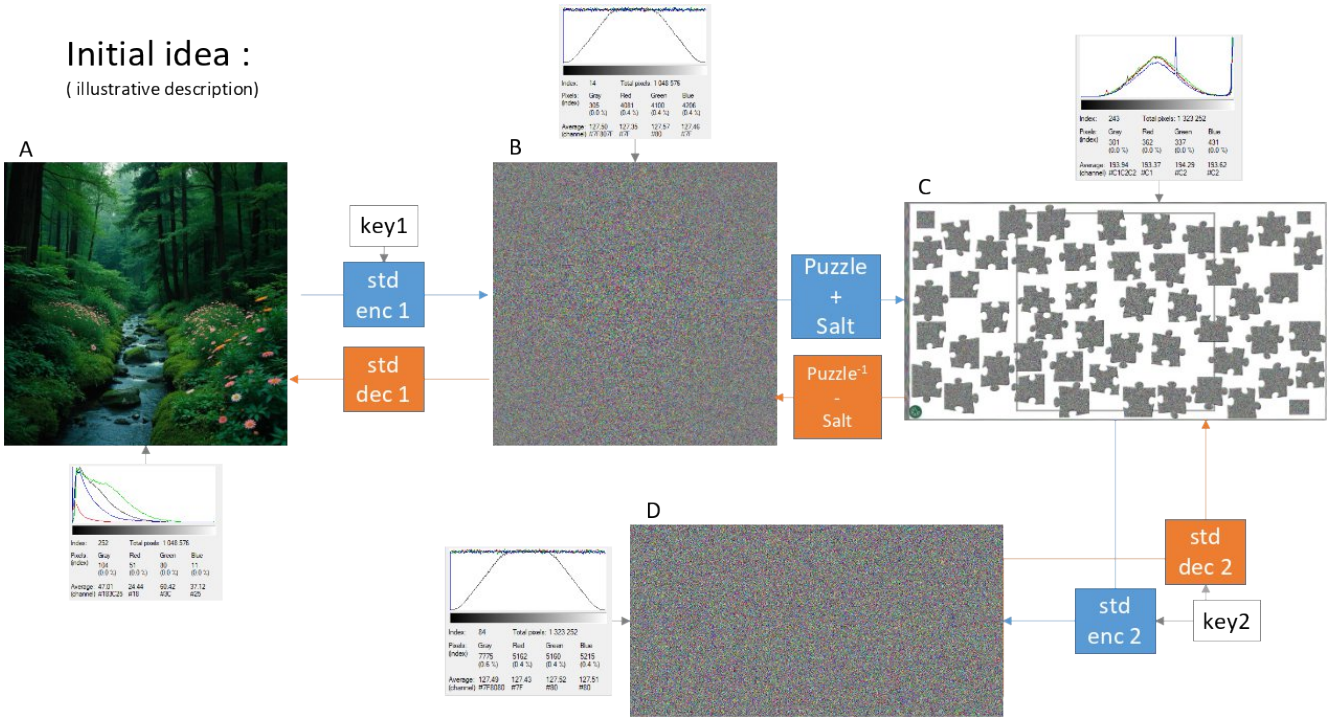

Figure 1 – Simplified PuzzleCrypt concept illustration (without the compression layer)

In addition to the Puzzle concept, we also take advantage of the properties of lossless data compression. The compression layer introduces an additional dependency on the original ordering of the data. In dictionary-based compression schemes such as LZ-family algorithms or Huffman coding, a significant part of the information required to reconstruct the message is contained in the compression context and the associated dictionary. In addition words are encoded with variable size.

In PuzzleCrypt, the compressed stream is fragmented and permuted before being protected by an additional encryption layer. As a result, the attacker may not know where the beginning of the compressed stream is located, nor where the critical dictionary information resides. Recovering all fragments is therefore not necessarily sufficient; the correct reconstruction of the dictionary and its position within the data stream becomes an essential part of solving the puzzle. Consequently, the problem is transformed from a pure decryption task into a combined decryption, reordering, and reconstruction task. The underlying hypothesis is that recovering encrypted fragments may be easier than recovering the contextual information required to interpret and decompress them correctly. Our solution does not assume that fragmentation prevents the decryption of individual blocks. Rather, it investigates whether the use of variable-sized fragments and deliberate loss of ordering information can increase the overall complexity of recovering useful information from decrypted fragments. Under this hypothesis, an adversary may face a two-stage challenge: first recovering the contents of individual fragments, and then reconstructing their original size, organization and context.

3. PuzzleCrypt Architecture

PuzzleCrypt separates data recovery from message recovery. An attacker may successfully recover a collection of valid fragments while still lacking the critical contextual information required to reconstruct the compressed stream, particularly when the compression dictionary and its position within the stream are unknown. This transforms decryption into a combined decryption-and-reconstruction problem rather than a purely cryptographic one.

```
Recovering fragments
≠
Recovering message
```

3.1 Encryption Process

The process, which combines compression, encryption, and block mixing, consists of the following steps:

```
Original File
      ↓
AES-256 (password 1) + LZ Compression
      ↓
inner.7z
      ↓
Block Splitting
      ↓
SHA256-Based Permutation
       ↓
 Permuted Pieces
       ↓
 AES-256 (password 2) + LZ Compression
       ↓
 PuzzleCrypt Archive
```

3.2 Decryption Process

The reverse process used to recover the secret information involves the following steps:

```
AES-256 (password 2) + Unzip
     ↓
Recover Pieces
     ↓
Inverse Permutation
     ↓
Reconstruct inner.7z
     ↓
AES-256 (password 1) + Unzip
      ↓
 Original File
```

3.3 Implementation

Mention:

```
Python
py7zr
```

```
3  AES-256
4  SHA256
```

Current prototype:

```
1  PuzzleCrypt v0
```

4. Preliminary Experimental Results

This first simple version of PuzzleCrypt (that can be found here [12]) is implemented as a fragmentation and permutation layer applied on top of a standard AES-based encryption workflow. The input file is first compressed and encrypted using 7-Zip (AES-256), then divided into multiple fragments. These fragments are deterministically permuted according to a key-derived permutation scheme and stored within the PuzzleCrypt archive. During decryption, the inverse permutation is applied to restore the original fragment order before standard AES decryption and decompression recover the source file. Since the confidentiality of the protected content ultimately relies on the security guarantees of AES, the objective of the experimental evaluation is not to re-establish cryptographic security, but rather to verify that the PuzzleCrypt processing pipeline preserves these guarantees while providing controlled data fragmentation and dispersion. The evaluation therefore focuses on demonstrating the practical feasibility of the architecture. Thus we show a successful encryption/decryption illustrated on Figure 2, validating lossless reconstruction of the original data. The encrypted archive size is about the same as the initial fianles in amny cases so the storage overhead introduced by the PuzzleCrypt layer is minor. In practice the number of fragments will vary from 10 to 1024 (set as a maximum). This number depends of a caclulation of the file size divided by the block size. Collectively, these results indicate that PuzzleCrypt maintains at least the security level provided by the underlying AES encryption while introducing an additional organizational layer based on fragmentation and permutation. Nevertheless, we conducted image encryption tests that appear to be compliant: for 10 encrypted images, the mean of the distribution is around 128/255, with a standard deviation of around 70 and a flat distribution in all cases. This is to be expected, since AES-256 produces these results and is used twice in our algorithm.

In addition to the sample encryption tests, we have also uploaded password-less encryption examples to the project's GitHub repository [12] in case anyone or any machine manages to break this version of the algorithm (which could then be made even more complex).

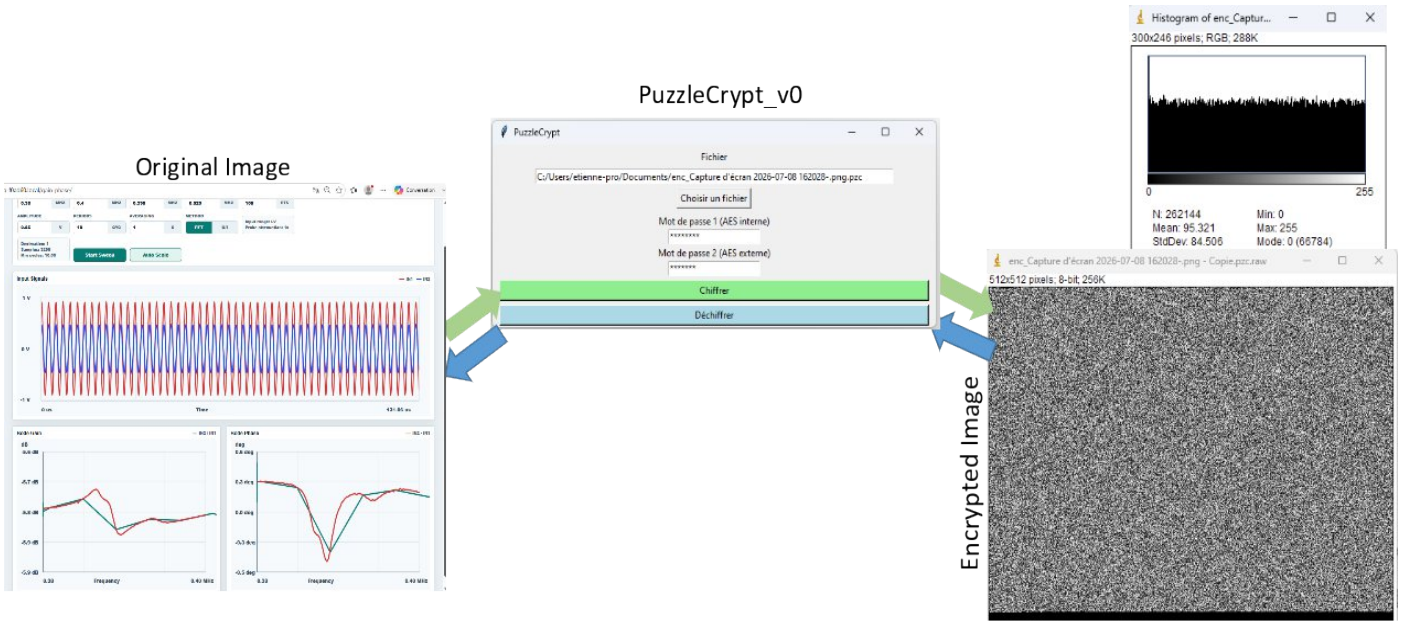

Figure 2 - PuzzleCrypt prototype illustration, original image, PuzzleCrypt HMI interface, resulted encrypted image and its distribution.

5. Limitations

PuzzleCrypt is currently at the concept and early prototype stage. Although the implementation demonstrates successful operation and compatibility with standard AES-based encryption, no formal security proof, cryptographic reduction, quantum-complexity proof, or comprehensive attack analysis has yet been established. Consequently, the present results should be interpreted as an engineering validation of the proposed architecture rather than a definitive assessment of its cryptographic security.

6. Discussion and perspectives

PuzzleCrypt is not intended to provide information-theoretic security, nor is it claimed to offer proven resistance against classical or quantum cryptanalytic attacks. The confidentiality of the protected data remains fundamentally dependent on the security of the underlying AES encryption layers. Instead, the objective of PuzzleCrypt is to explore a different research direction: whether the combination of fragmentation, permutation, and loss of ordering information can transform a purely cryptographic problem into a combined cryptographic-and-reconstruction problem. This distinction is important. In conventional encrypted archives, an adversary is primarily confronted with the challenge of recovering the encryption key and decrypting the protected data. In PuzzleCrypt, successful recovery may additionally require identifying the correct arrangement of fragmented data and reconstructing the original structure before meaningful information can be obtained.

The motivation for this approach is partially inspired by the uncertainty surrounding future quantum cryptanalytic capabilities. This work does not claim any form of quantum resistance. Rather, it investigates whether the fragmentation process could alter the practical characteristics of a future attack.

Two working assumptions motivate this research:

Assumption A: Future quantum processors may become capable of efficiently attacking or analyzing encrypted data when presented as bounded-size, well-structured ciphertext blocks.

Assumption B: Any potential quantum advantage may be reduced if the attacker must repeatedly perform additional reconstruction tasks involving fragmented, reordered, or partially context-free data.

Under this hypothesis, the attack workflow evolves from a relatively direct process:

[Plain Text => Decrypt]

to a more complex iterative process:

[Plain Text => Decrypt => Reorder => Reconstruct/deflate => Decrypt => Reconstruct/deflate]

The contribution of PuzzleCrypt is therefore not to replace cryptographic security, but to explore whether introducing a reconstruction layer can increase the practical complexity faced by an attacker. This leads to the central research question motivating future investigations: « Does progressive loss of ordering information reduce the practical advantage of future quantum cryptanalytic systems by coupling decryption with a non-trivial reconstruction process? »

At present, no theoretical framework or experimental evidence is sufficient to answer this question definitively. The architecture should therefore be viewed as an exploratory research platform rather than a cryptographic scheme with established security guarantees.

Several extensions of the PuzzleCrypt architecture could be investigated to further increase the complexity of the reconstruction process and evaluate its impact on practical attack scenarios. (1)The current prototype stores only genuine fragments originating from the encrypted file. A future version could introduce noise blocks that are indistinguishable from authentic fragments without knowledge of the reconstruction parameters (ex : Real fragment, Fake fragment, Real fragment, ...). (2)Fragmentation could be applied recursively, creating multiple nested layers of reconstruction. This concept is illustrated on Figure 3. In such a design, recovering the correct arrangement at one level would reveal the information necessary to solve the next level. Rather than using a uniform fragmentation strategy, future

implementations could generate fragments of varying size. This would reduce structural regularity and potentially complicate automated reconstruction attempts. (3) Instead of representing the archive as a simple sequence of fragments, future PuzzleCrypt variants could model relationships between fragments as a graph. Reconstruction would then involve solving a graph recovery problem in addition to decryption. (4) The current prototype relies on linear fragmentation. Future systems could explore irregular fragment boundaries and non-sequential relationships between pieces, producing puzzle-like structures whose reconstruction is not naturally ordered. Such approaches may further increase the separation between successful decryption and successful recovery of meaningful data.

Overall, these directions aim to investigate whether fragmentation can evolve from a storage mechanism into an additional computational layer whose reconstruction complexity complements, without replacing, the security provided by conventional cryptographic primitives.

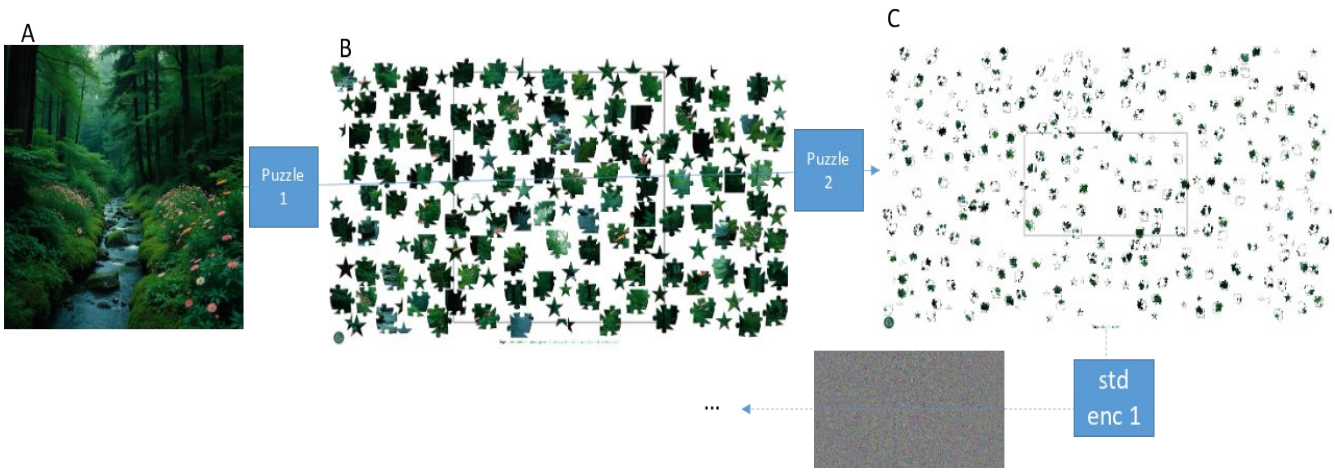

Figure 3 – Graphical illustration of multi-level puzzles, with some encryption steps.

7. Conclusion

This work introduced PuzzleCrypt, a proof-of-concept architecture that combines conventional AES-based encryption with a compression, fragmentation and permutation layer designed to separate confidentiality from structural recoverability. Rather than proposing a new cryptographic primitive, PuzzleCrypt investigates whether the deliberate loss of ordering information can transform data recovery into a joint problem involving both

cryptographic decryption and archive reconstruction. A functional prototype was successfully implemented and evaluated. Experimental results demonstrate the correctness of the encryption and decryption workflow, the lossless reconstruction of protected files, and the practical feasibility of the proposed fragmentation-permutation mechanism. Statistical analyses indicate that the additional processing stages preserve the properties expected from the underlying AES-encrypted data, while introducing controllable data dispersion through fragment reorganization.

The primary contribution of this work is therefore conceptual rather than cryptographic. PuzzleCrypt explores the possibility of augmenting traditional encryption with a reconstruction layer in which access to encrypted fragments alone is insufficient to immediately recover meaningful information without recovering their original organization. From this perspective, the approach shifts part of the recovery effort from a purely cryptographic objective toward a combined cryptographic-and-structural reconstruction problem. No claim is made that PuzzleCrypt provides information-theoretic security, formal quantum resistance, or security beyond that of the underlying AES implementation. Likewise, no formal security proof, cryptographic reduction, or comprehensive attack analysis has yet been established. Consequently, the present study should be viewed as an initial exploration of a new design space rather than as a finalized cryptographic scheme.

Looking forward, the most important research question remains whether progressive loss of ordering information can measurably increase the practical complexity of future attack strategies, particularly in scenarios where quantum decryption must be coupled with fragment identification, validation, and reconstruction. In summary, PuzzleCrypt demonstrates that fragmentation and permutation can be integrated into a practical encryption workflow while preserving the security guarantees of standard cryptographic primitives. Whether such mechanisms can provide meaningful additional resistance against future adversarial capabilities remains an open and promising direction for further research.

8. Bibliography

[1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994.

[2] R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” Communications of the ACM, vol. 21, no. 2, pp. 120–126, 1978.

[3] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” Proc. 28th Annual ACM Symposium on Theory of Computing (STOC), 1996.

[4] S. D. and P. C., “On the Practical Cost of Grover for AES Key Recovery,” NCSC / NIST PQC Conference Paper, 2024. 【1-d64aa7】

[5] National Institute of Standards and Technology (NIST), Post-Quantum Cryptography Standardization Project, 2016–2024.

[6] J. Rizzo and T. Duong, “The CRIME Attack,” Ekoparty Security Conference, 2012.

[7] Y. Gluck, N. Harris, and A. Prado, “BREACH: Reviving the CRIME Attack,” Black Hat USA, 2013.

[8] A. Shamir, “How to Share a Secret,” Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[9] M. O. Rabin, “Efficient Dispersal of Information for Security, Load Balancing, and Fault Tolerance,” Journal of the ACM, vol. 36, no. 2, pp. 335–348, 1989.

[10] R. L. Rivest, “All-Or-Nothing Encryption and The Package Transform,” Fast Software Encryption (FSE), 1997.

[11] Z. Wilcox-O’Hearn and B. Warner, “Tahoe: The Least-Authority Filesystem,” ACM Workshop on Storage Security and Survivability, 2008.

[12] PuzzleCrypt project repository & source code:
https://github.com/elunaire/PuzzleScript
https://github.com/elunaire/PuzzleScript/blob/main/PuzzleCryptGUI_v0.py